

1ª AVALIAÇÃO
Trabalho de Implementação — Jogo da Força

1. Contextualização

O Jogo da Força é um jogo tradicional no qual o jogador deve descobrir uma **resposta oculta**, formada por uma ou mais palavras, por meio da escolha sucessiva de letras. A cada tentativa incorreta, uma nova parte da representação da força é exibida. O jogador vence quando identifica todas as letras da resposta antes de atingir o limite máximo de erros permitido.

Embora apresente regras simples e amplamente conhecidas, sua implementação envolve diferentes problemas de programação, incluindo manipulação de strings, coleções, gerenciamento de estado, sorteio de elementos, validação de regras de negócio, leitura de dados estruturados e atualização dinâmica da interface.

A proposta deste trabalho é desenvolver, utilizando **Flutter e Dart**, uma versão completa do Jogo da Força para dispositivos móveis.

A aplicação deverá funcionar integralmente de forma **offline**. As respostas, categorias, níveis de dificuldade, dicas e demais informações necessárias ao funcionamento do jogo serão fornecidas juntamente com este enunciado em um **arquivo JSON**, que deverá ser incorporado aos assets da aplicação.

Uma resposta poderá ser constituída por:

- uma única palavra;
- duas ou mais palavras;
- palavras separadas por espaços;
- termos contendo hífen ou outros caracteres não alfabéticos presentes nos dados fornecidos.

Espaços, hífen e caracteres similares que façam parte da resposta deverão ser apresentados automaticamente na interface. O jogador deverá realizar tentativas **exclusivamente por meio da seleção de letras**, não sendo necessário nem permitido digitar ou selecionar esses caracteres.

Não deverá ser utilizado banco de dados, serviço web, API externa ou qualquer outro mecanismo de armazenamento ou comunicação remota.

2. Requisitos Funcionais

RF01 — Início do jogo

O aplicativo deverá permitir ao usuário iniciar uma nova rodada selecionando:

- uma categoria;
- um nível de dificuldade.

As categorias e os níveis disponíveis deverão ser obtidos a partir dos dados fornecidos no arquivo JSON.

Antes do início da rodada, o aplicativo deverá apresentar claramente as opções selecionadas.

RF02 — Categorias e níveis de dificuldade

As respostas deverão estar organizadas em categorias e níveis de dificuldade conforme os dados fornecidos.

A aplicação deverá:

- apresentar apenas categorias existentes no conjunto de dados;
- permitir selecionar o nível de dificuldade;
- selecionar somente respostas compatíveis com a categoria e o nível escolhidos;
- utilizar o arquivo JSON fornecido como fonte de dados para respostas e dicas.

A equipe não deverá alterar o significado das categorias ou dos níveis definidos no conjunto de dados.

RF03 — Sorteio da resposta

Ao iniciar uma rodada, uma resposta compatível com a categoria e o nível escolhidos deverá ser selecionada aleatoriamente.

A resposta poderá conter **uma ou mais palavras**.

As letras deverão permanecer ocultas inicialmente. Espaços, hífen e demais caracteres não alfabéticos presentes na resposta deverão, entretanto, ser exibidos automaticamente desde o início da rodada.

Exemplo com uma única palavra:

Resposta: COMPUTADOR

Exibição inicial:

- - - - -

Exemplo com mais de uma palavra:

Resposta: COLUNA VERTEBRAL

Exibição inicial:

- - - - -

Exemplo contendo hífen:

Resposta: QUATI SUL-AMERICANO

Exibição inicial:

- - - - - - - -

Os espaços entre palavras, hífen e caracteres similares deverão permanecer visíveis durante toda a rodada.

Esses caracteres **não deverão ser digitados ou selecionados pelo jogador**.

RF04 — Não repetição de respostas

Uma resposta já utilizada não poderá ser sorteada novamente durante a mesma sessão da aplicação enquanto houver outras respostas disponíveis para a combinação de categoria e nível selecionada.

A aplicação deverá controlar as respostas já utilizadas durante a sessão.

Quando todas as respostas disponíveis para determinada combinação de categoria e nível tiverem sido utilizadas, a aplicação deverá informar essa condição ao usuário e tratá-la adequadamente.

Não é permitido repetir silenciosamente respostas já apresentadas.

RF05 — Tentativa de letras

O jogador deverá realizar suas tentativas selecionando **exclusivamente letras do alfabeto** disponibilizadas pela própria interface da aplicação.

Para cada letra escolhida:

- se a letra estiver presente na resposta, todas as suas ocorrências deverão ser reveladas;
- se a letra não estiver presente na resposta, deverá ser contabilizado um erro;
- uma letra já escolhida não poderá produzir uma nova tentativa;
- a interface deverá permitir identificar quais letras já foram utilizadas.

Quando a resposta possuir mais de uma palavra, a tentativa deverá ser aplicada à **resposta completa**, e não separadamente a cada palavra.

Por exemplo, se uma mesma letra ocorrer em duas palavras da expressão sorteada, todas as ocorrências deverão ser reveladas simultaneamente.

Espaços, hífen e demais caracteres não alfabéticos **não fazem parte do conjunto de tentativas possíveis**. Eles deverão ser apresentados diretamente pela interface e não deverão ser digitados ou selecionados pelo jogador.

A equipe poderá definir livremente a organização visual do teclado ou do conjunto de letras.

RF06 — Tratamento de acentuação e caracteres não alfabéticos

A comparação entre a letra escolhida pelo jogador e as letras da resposta deverá **ignorar acentuação**.

Assim:

- A deverá corresponder a A , Á , À , Â e Ã ;
- E deverá corresponder a E , É e Ê ;
- I deverá corresponder a I e Í ;
- O deverá corresponder a O , Ó , Ô e Õ ;
- U deverá corresponder a U e Ú .

A resposta exibida ao usuário deverá, entretanto, **preservar sua grafia original**.

Exemplo:

Resposta: CORAÇÃO

Tentativa: A

Nesse caso, a ocorrência correspondente a ã deverá ser revelada, permanecendo exibida como ã .

A normalização utilizada exclusivamente para comparação não deverá modificar a representação original da resposta.

Espaços e demais caracteres não alfabéticos

Caracteres que não correspondam a letras deverão ser apresentados automaticamente na interface desde o início da rodada.

Isso inclui, quando presentes na resposta:

- espaços;
- hífen;
- apóstrofes;
- outros sinais de pontuação existentes nos dados fornecidos.

Esses caracteres:

- deverão permanecer visíveis durante toda a rodada;
- **não deverão ser digitados pelo jogador;**
- **não deverão ser disponibilizados como opções de tentativa;**
- não deverão ser contabilizados como letras ocultas;
- não deverão gerar erros;
- não deverão interferir na condição de vitória.

O jogador deverá interagir apenas com as letras disponibilizadas pela interface.

RF07 — Controle de erros

A aplicação deverá definir um número máximo de erros permitidos para cada rodada.

Cada tentativa incorreta deverá:

- incrementar o número de erros;
- atualizar a representação visual da força;
- atualizar as informações apresentadas ao jogador.

O jogador perderá a rodada quando atingir o número máximo de erros permitido.

RF08 — Representação gráfica progressiva da força

O aplicativo deverá apresentar uma representação gráfica progressiva da força e do boneco.

A representação deverá evoluir conforme o número de erros cometidos.

O boneco deverá ser formado, no mínimo, pelas seguintes partes:

1. cabeça;
2. tronco;
3. braço esquerdo;
4. braço direito;
5. perna esquerda;
6. perna direita.

A estrutura da força poderá estar presente desde o início da rodada ou fazer parte da sequência visual adotada pela equipe, desde que a progressão dos erros seja claramente perceptível.

Cada novo erro deverá produzir uma alteração visual correspondente.

A equipe poderá utilizar:

- imagens armazenadas nos assets;
- composição de widgets;
- ícones;
- desenho programático.

Não será exigida a utilização de `CustomPainter`.

O limite máximo de erros deverá ser compatível com a quantidade de estados gráficos implementados.

RF09 — Sistema de dicas

Cada resposta possuirá uma ou mais dicas no arquivo JSON fornecido.

Durante a rodada, o jogador deverá poder solicitar uma dica relacionada à resposta atual.

O sistema deverá:

- apresentar a dica ao jogador;
- controlar se uma dica já foi utilizada naquela rodada;
- contabilizar sua utilização nas estatísticas da sessão.

A consulta repetida à mesma dica durante uma rodada não deverá produzir contabilizações adicionais.

RF10 — Condição de vitória

O jogador vencerá a rodada quando todas as **letras da resposta** tiverem sido corretamente identificadas antes de atingir o limite máximo de erros.

Espaços, hífen e demais caracteres não alfabéticos não deverão participar da verificação da condição de vitória, pois já deverão estar visíveis desde o início da rodada.

Ao final de uma rodada vencida, a aplicação deverá apresentar, no mínimo:

- a resposta completa;
- indicação de vitória;
- quantidade de erros;
- indicação de utilização ou não de dica.

RF11 — Condição de derrota

O jogador perderá a rodada quando atingir o limite máximo de erros.

Nesse caso, a aplicação deverá:

- informar explicitamente a derrota;
- revelar a resposta completa;

- apresentar a representação final da força;
- permitir iniciar uma nova rodada.

RF12 — Nova rodada

Após o encerramento de uma rodada, o usuário deverá poder iniciar outra.

A aplicação deverá manter durante a sessão:

- as estatísticas acumuladas;
- o controle das respostas já utilizadas.

Também deverá ser possível retornar à seleção de categoria e nível.

RF13 — Estatísticas da sessão

A aplicação deverá manter estatísticas enquanto permanecer em execução.

Deverão ser apresentadas, no mínimo:

- número total de rodadas realizadas;
- número e percentual de vitórias;
- número e percentual de derrotas;
- quantidade total de erros cometidos;
- quantidade de dicas utilizadas.

Os percentuais deverão ser calculados a partir das rodadas concluídas.

Os dados poderão ser reinicializados quando a aplicação for encerrada.

Não deverá ser implementada persistência dessas estatísticas em banco de dados ou serviço externo.

A interface deverá possuir uma forma clara de consultar as estatísticas da sessão.

RF14 — Identificação dos membros da equipe

A aplicação deverá possuir uma funcionalidade que permita visualizar os nomes completos dos integrantes da equipe responsável pelo desenvolvimento.

Essa informação poderá ser apresentada, por exemplo, em:

- tela específica;
- opção "Sobre";
- caixa de diálogo;
- seção equivalente.

Os nomes deverão estar acessíveis ao usuário a partir da interface da aplicação.

3. Dados fornecidos em JSON

As informações necessárias para execução do jogo serão disponibilizadas juntamente com este enunciado em um arquivo JSON.

O conjunto de dados utiliza o conceito de **resposta** (`answer`), pois uma entrada poderá representar uma única palavra ou uma expressão composta por várias palavras.

O arquivo conterá informações relacionadas a:

- resposta;
- categoria;
- nível de dificuldade;
- dica ou dicas associadas.

A estrutura seguirá o formato geral abaixo:

```
{
  "entries": [
    {
      "answer": "COLUNA VERTEBRAL",
      "category": "humanBody",
      "difficulty": "medium",
      "hints": [
        "Estrutura óssea que sustenta o tronco e protege parte do sistema nervoso."
      ]
    },
    {
      "answer": "SISTEMA OPERACIONAL",
      "category": "technology",
      "difficulty": "medium",
      "hints": [
        "Software responsável por gerenciar recursos do computador e oferecer serviços a outros programas."
      ]
    }
  ]
}
```

O campo `answer` deverá ser tratado como a resposta completa da rodada.

Uma `answer` poderá conter:

- uma única palavra;
- duas ou mais palavras;
- espaços;
- caracteres acentuados;
- hífen ou outros caracteres não alfabéticos.

A aplicação não deverá assumir que uma resposta corresponde obrigatoriamente a uma única palavra.

Espaços, hífen e caracteres similares presentes no valor de `answer` deverão ser preservados e apresentados automaticamente ao jogador. Esses caracteres **não deverão ser digitados ou selecionados durante a partida**.

3.1. Valores enumerados obrigatórios

Os campos `category` e `difficulty` possuem conjuntos fechados de valores e deverão ser obrigatoriamente representados no código Dart por meio de tipos enumerados (`enum`).

Os únicos valores válidos para o campo `category` são:

```
animals
humanBody
sports
technology
```

A aplicação deverá possuir uma representação equivalente no domínio, por exemplo:

```
enum AnswerCategory {
  animals,
  humanBody,
  sports,
  technology,
}
```

Os únicos valores válidos para o campo `difficulty` são:

```
easy
medium
hard
```

A aplicação deverá possuir uma representação equivalente:

```
enum Difficulty {
  easy,
  medium,
  hard,
}
```

Durante a desserialização do JSON, os valores textuais desses campos deverão ser convertidos para os respectivos tipos enumerados.

Não é permitido manter `category` ou `difficulty` como `String` no modelo de domínio da aplicação.

Caso seja necessário apresentar esses valores na interface, a conversão para textos legíveis em português deverá ser realizada pela aplicação.

Valor técnico	Texto apresentado ao usuário
animals	Animais
humanBody	Corpo humano
sports	Esportes
technology	Tecnologia
easy	Fácil
medium	Médio
hard	Difícil

Os valores técnicos presentes no JSON e utilizados internamente no código deverão permanecer em inglês. Os textos apresentados ao usuário deverão estar em português.

A exigência de uso de tipos enumerados aplica-se também a outros atributos do domínio que representem conjuntos fechados e previamente conhecidos de valores.

3.2. Carregamento e desserialização

O arquivo JSON deverá ser incluído no projeto como um **asset da aplicação**, sendo declarado no `pubspec.yaml`.

Exemplo:

```
flutter:
  assets:
    - assets/data/hangman_entries.json
```

O conteúdo deverá ser carregado em tempo de execução. Uma possibilidade é utilizar `rootBundle`:

```
import 'package:flutter/services.dart';

final jsonString =
  await rootBundle.loadString('assets/data/hangman_entries.json');
```

A partir da `String` carregada, a equipe deverá realizar a desserialização do JSON e convertê-lo em objetos Dart adequadamente modelados.

A equipe deverá implementar:

1. carregamento do asset;

2. desserialização do JSON;
3. conversão das entradas para objetos Dart;
4. conversão dos campos com valores predefinidos para os respectivos tipos enumerados;
5. representação das respostas e demais dados por meio de classes adequadas;
6. tratamento apropriado de erros ou inconsistências durante o carregamento.

Não é permitido copiar manualmente o conteúdo do arquivo JSON para listas, mapas, constantes ou objetos definidos diretamente no código-fonte.

A aplicação deverá obrigatoriamente carregar e processar, em tempo de execução, o arquivo JSON fornecido com a atividade.

Implementações que reproduzam os dados do arquivo diretamente no código-fonte, evitando seu carregamento e desserialização, serão consideradas como **não atendimento deste requisito**, ainda que apresentem comportamento funcional equivalente.

4. Regras de negócio obrigatórias

As seguintes regras deverão ser satisfeitas independentemente das decisões de interface adotadas pela equipe:

1. uma letra correta deverá revelar todas as suas ocorrências na resposta, inclusive quando estiver presente em palavras diferentes da mesma expressão;
2. uma letra incorreta deverá gerar somente um erro;
3. selecionar novamente uma letra já utilizada não deverá produzir novo erro;
4. a comparação entre letras deverá ignorar acentuação;
5. a grafia original da resposta deverá ser preservada na interface;
6. espaços, hífen e demais caracteres não alfabéticos deverão ser exibidos automaticamente desde o início da rodada;
7. espaços, hífen e demais caracteres não alfabéticos não deverão ser digitados ou selecionados pelo jogador;
8. somente letras deverão constituir tentativas válidas;
9. caracteres não alfabéticos não deverão interferir na condição de vitória;
10. respostas não deverão ser repetidas enquanto houver respostas ainda não utilizadas na mesma categoria e nível;
11. vitória e derrota são estados mutuamente exclusivos;
12. nenhuma tentativa deverá ser aceita depois do encerramento da rodada;
13. a utilização da dica deverá ser registrada no máximo uma vez por rodada;
14. as estatísticas deverão refletir corretamente todas as rodadas concluídas.

5. Requisitos Não Funcionais

RNF01 — Usabilidade

A interface deverá:

- ser intuitiva;
- apresentar claramente o estado atual da rodada;
- distinguir letras disponíveis das já utilizadas;
- representar adequadamente respostas constituídas por uma ou mais palavras;
- permitir identificar visualmente os limites entre as palavras de uma expressão;
- apresentar automaticamente espaços, hífen e caracteres similares;
- fornecer feedback visual após cada tentativa;
- deixar evidentes as condições de vitória e derrota.

RNF02 — Interface responsiva

A aplicação deverá funcionar adequadamente em diferentes dimensões de tela de dispositivos Android.

Não deverão ocorrer:

- elementos sobrepostos;
- textos cortados;
- controles inacessíveis;
- erros de overflow;
- dependência de uma resolução específica.

Respostas longas ou constituídas por várias palavras deverão ser apresentadas de forma legível, sem provocar overflow ou comprometer a utilização da interface.

RNF03 — Qualidade visual

Cada equipe possui liberdade para definir a identidade visual da aplicação.

Serão considerados na avaliação:

- organização;
- legibilidade;
- consistência visual;
- hierarquia das informações;
- uso adequado dos componentes do Flutter;
- qualidade geral da experiência oferecida.

Recursos gráficos não deverão comprometer a usabilidade da aplicação.

RNF04 — Idioma, nomenclatura e estilo de código

A interface da aplicação, a documentação textual e os demais conteúdos destinados à leitura humana deverão estar em **português**, salvo quando o próprio contexto técnico exigir outra forma.

Deverão estar em **inglês** os elementos técnicos de identificação utilizados no desenvolvimento, incluindo:

- classes;
- métodos e funções;
- variáveis e propriedades;
- tipos enumerados e respectivos valores;
- identificadores internos;
- nomes de arquivos de código;
- nomes de diretórios criados pela equipe;
- nomes técnicos de assets;
- nomes de branches;
- nome do repositório;
- componentes técnicos de URLs.

Exemplos:

```
hangman-game
answer_repository.dart
game_screen.dart
assets/data/hangman_entries.json
feature/game
```

Respostas do jogo, dicas, mensagens da interface, documentação e demais textos destinados a pessoas poderão permanecer em português.

A codificação deverá seguir as recomendações do **Effective Dart**:

<https://dart.dev/effective-dart>

RNF05 — Qualidade do código

O projeto deverá:

- utilizar código organizado e legível;
- evitar duplicação desnecessária;
- utilizar nomes expressivos;
- decompor adequadamente widgets e responsabilidades;
- empregar `const` sempre que pertinente;
- separar adequadamente lógica de negócio e interface.

O comando:

```
flutter analyze
```

deverá ser executado sem erros ou advertências provenientes do código desenvolvido pela equipe.

O código deverá estar formatado utilizando:

```
dart format .
```

RNF06 — Funcionamento offline

A aplicação deverá funcionar integralmente sem acesso à Internet.

Não poderão ser necessários serviços externos para iniciar, executar ou utilizar qualquer funcionalidade obrigatória.

6. Modelagem e organização da solução

As regras do jogo não deverão ficar integralmente acopladas à interface gráfica.

Espera-se separação adequada entre:

- representação dos dados;
- regras do jogo;
- estado da rodada;
- estatísticas da sessão;
- carregamento dos dados;
- widgets responsáveis pela apresentação.

Classes e tipos enumerados deverão ser utilizados sempre que representarem adequadamente conceitos do domínio, tais como:

- resposta;
- categoria;
- dificuldade;
- estado da rodada;
- estatísticas.

A modelagem deverá considerar explicitamente que uma resposta pode representar **uma palavra ou uma expressão composta por várias palavras**.

Também deverá diferenciar adequadamente as **letras que constituem tentativas do jogador** dos espaços, hífen e demais caracteres estruturais da resposta, que deverão ser apenas apresentados pela interface.

A organização arquitetural fica a critério da equipe, desde que resulte em código organizado, compreensível, de fácil manutenção e coerente com os critérios estabelecidos neste enunciado.

7. Restrições técnicas

Não deverão ser utilizados:

- bancos de dados;
- Firebase ou serviços equivalentes;
- APIs REST;
- APIs externas;
- acesso à Internet;
- autenticação de usuários;
- armazenamento remoto;
- serviços de backend.

Os dados necessários ao jogo deverão estar contidos no arquivo JSON fornecido e nos demais recursos locais do aplicativo.

Pacotes externos somente deverão ser utilizados quando tecnicamente justificados.

A utilização de pacote que implemente diretamente as regras centrais do Jogo da Forca poderá resultar em desconsideração dos respectivos requisitos na avaliação.

8. Funcionalidades e telas mínimas

A organização das telas é uma decisão de projeto da equipe. Entretanto, a aplicação deverá fornecer acesso claro às seguintes funcionalidades.

8.1. Configuração da partida

Deverá permitir:

- selecionar categoria;
- selecionar dificuldade;
- iniciar a rodada.

8.2. Tela principal do jogo

Deverá apresentar, no mínimo:

- categoria;
- dificuldade;
- resposta parcialmente revelada;
- separação visual adequada entre as palavras, quando houver mais de uma;
- espaços, hífen e demais caracteres similares existentes na resposta;
- letras disponíveis;
- letras já utilizadas;
- representação da força;
- quantidade ou progresso dos erros;
- acesso à dica.

Os controles de tentativa deverão disponibilizar somente letras. Espaços, hífen e demais caracteres similares deverão ser apresentados como parte da resposta, sem qualquer ação do jogador.

8.3. Resultado da rodada

Deverá indicar:

- vitória ou derrota;
- resposta completa;
- quantidade de erros;
- utilização de dica;
- possibilidade de iniciar nova rodada.

8.4. Estatísticas

Deverá apresentar as estatísticas acumuladas durante a sessão.

8.5. Identificação da equipe

Deverá permitir consultar os nomes completos dos integrantes responsáveis pelo desenvolvimento da aplicação.

9. Composição da equipe

- A implementação deverá ser realizada em equipes de **4 (quatro) estudantes**.
- Todos os integrantes são responsáveis pelo conhecimento integral da solução entregue.
- **Cada integrante será questionado individualmente** durante a sessão de apresentação da equipe, inclusive sobre partes do projeto implementadas originalmente por outros integrantes.
- Todos os membros deverão ser capazes de explicar as principais decisões de projeto, regras de negócio, modelagem, arquitetura e código-fonte.
- A divisão interna de tarefas não elimina a responsabilidade de cada integrante pelo conhecimento da solução completa.
- O histórico do repositório deverá demonstrar participação efetiva de todos os membros.
- A participação de todos os integrantes na sessão de apresentação é **obrigatória**.

A ausência de um integrante na sessão de apresentação implicará **pontuação 0 para esse integrante na composição da nota da apresentação da equipe**.

Essa pontuação zero será considerada normalmente no cálculo da média da apresentação, não implicando, por si só, nota zero para toda a equipe ou para o trabalho completo.

10. Documentação do projeto

O projeto deverá possuir um arquivo `README.md` compatível com o que normalmente se espera de um repositório público de software.

No início do `README.md` deverão constar, obrigatoriamente, os nomes completos de todos os integrantes da equipe.

Após essa identificação, o documento deverá conter, no mínimo:

- nome do projeto;
- breve descrição;
- principais funcionalidades;
- tecnologias utilizadas;
- instruções para obtenção e execução do projeto;
- eventuais dependências adicionais;
- estrutura geral da aplicação, quando pertinente.

A ausência da identificação dos integrantes no início do `README.md` caracterizará descumprimento deste requisito.

11. Repositório Git

O projeto deverá ser mantido em um repositório **público** no GitHub.

O repositório deverá:

- possuir `main` como branch principal;
- estar publicamente acessível no momento da entrega e durante todo o período de avaliação;
- apresentar histórico de commits coerente com o processo de desenvolvimento;
- possuir commits realizados por **todos os integrantes da equipe**;
- permitir identificar participação efetiva dos membros ao longo do desenvolvimento;
- conter o arquivo `README.md` exigido neste enunciado;
- conter apenas arquivos necessários ao desenvolvimento, compilação e execução do projeto.

A participação de todos os integrantes por meio de commits identificáveis no histórico do Git constitui **critério de avaliação**.

Os commits deverão representar contribuições efetivas. A criação artificial de commits apenas para registrar participação poderá ser desconsiderada.

O repositório não deverá conter artefatos gerados pelo processo de compilação, empacotamento ou execução nem arquivos que normalmente devam ser excluídos pelo `.gitignore`.

Isso inclui, quando aplicável:

- diretórios de build;
- arquivos APK;
- Android App Bundles (`.aab`);
- executáveis ou outros binários compilados;
- caches;
- arquivos temporários;
- arquivos intermediários produzidos pelas ferramentas;
- configurações locais de IDE que não façam parte do projeto;
- outros artefatos que possam ser reproduzidos a partir do código-fonte.

O `.gitignore` deverá estar adequadamente configurado para um projeto Flutter.

A inexistência do repositório público ou a impossibilidade de acesso ao endereço informado poderá inviabilizar a avaliação da entrega.

12. Entrega

A entrega deverá ser realizada exclusivamente pelo **SIGAA** até o dia **12/10/2026**.

Deverá ser submetido:

1. o link para o repositório público do projeto no GitHub;
2. imediatamente após o link, um comentário contendo os nomes completos dos integrantes da equipe.

Exemplo:

```
https://github.com/username/hangman-game
```

Integrantes:

- Nome completo do integrante 1
- Nome completo do integrante 2
- Nome completo do integrante 3
- Nome completo do integrante 4

O nome do repositório e demais elementos técnicos utilizados na URL deverão seguir a convenção de uso do inglês estabelecida neste enunciado.

O repositório informado deverá conter todos os arquivos necessários para obtenção, compilação e execução da aplicação, incluindo:

- código-fonte;

- `pubspec.yaml` ;
- arquivo JSON fornecido para a atividade;
- assets necessários;
- `README.md` ;
- arquivos de configuração indispensáveis ao projeto.

O repositório deverá conter o **código-fonte e os recursos necessários para reproduzir a aplicação**, e não os produtos gerados pelo processo de build.

Portanto, não deverão ser incluídos APKs, Android App Bundles, diretórios de build, binários, caches ou outros artefatos gerados que possam ser reconstruídos a partir do código-fonte.

A aplicação deverá poder ser obtida e executada a partir do repositório entregue sem necessidade de serviços externos.

13. Sistema de avaliação

A avaliação será realizada em **duas etapas**, cada uma contribuindo com até **5,0 pontos** para a nota final:

1. **entrega do código e avaliação da solução pelo professor** — até **5,0 pontos**;
2. **sessão de apresentação, demonstração e arguição da equipe** — até **5,0 pontos**.

A nota final do trabalho será expressa na escala de **0 a 10** e calculada da seguinte forma:

Nota final = nota da entrega + nota da apresentação

13.1. Escala de avaliação

Cada requisito, critério, resposta ou desempenho individual analisado receberá uma pontuação entre **0 e 5**, conforme o grau de atendimento:

Pontuação	Grau de atendimento
0	Não atende.
1 a 4	Atende parcialmente, em grau proporcional à correção, completude e qualidade demonstradas.
5	Atende completamente.

13.2. Etapa 1 — Entrega e avaliação da solução — até 5,0 pontos

Após a entrega pelo SIGAA, o professor avaliará o repositório e a aplicação submetida.

Cada requisito funcional, requisito não funcional e critério técnico aplicável será avaliado individualmente com pontuação entre **0 e 5**.

A **nota da entrega** será determinada pela média das pontuações atribuídas aos itens avaliados, resultando em uma nota entre **0 e 5**.

Serão considerados, entre outros:

- correção funcional;
- atendimento às regras de negócio;
- tratamento adequado de respostas compostas por uma ou mais palavras;
- apresentação e tratamento correto de espaços, hífen e demais caracteres não alfabéticos;
- restrição das tentativas do jogador às letras;
- tratamento de situações de contorno;
- carregamento e modelagem dos dados JSON;
- uso apropriado de tipos enumerados;
- qualidade e organização do código;
- interface, usabilidade e responsividade;
- modelagem e arquitetura;
- documentação;
- organização do repositório;
- histórico de commits;
- participação dos integrantes no processo de desenvolvimento.

A nota obtida nesta etapa será comum a todos os integrantes da equipe e contribuirá com até **5,0 pontos** para a nota final.

13.3. Qualidade e organização do código

Na avaliação da qualidade do código serão considerados:

- clareza;

- legibilidade;
- simplicidade;
- coesão;
- nomenclatura;
- aderência ao Effective Dart;
- ausência de duplicação desnecessária;
- decomposição adequada de widgets e responsabilidades;
- utilização apropriada dos recursos de Dart e Flutter;
- análise estática sem problemas;
- aplicação coerente dos princípios de **Clean Code**.

13.4. Modelagem e arquitetura

Será considerada a qualidade da representação dos conceitos do domínio e da organização estrutural da aplicação.

Serão avaliados, entre outros aspectos:

- modelagem adequada da resposta, incluindo respostas constituídas por várias palavras;
- tratamento adequado de letras e caracteres não alfabéticos;
- uso apropriado de classes e tipos enumerados;
- separação entre interface, estado, regras de negócio e acesso aos dados locais;
- dependências coerentes entre os componentes;
- baixo acoplamento;
- alta coesão;
- aplicação pertinente dos princípios **SOLID**;
- adoção de princípios compatíveis com **Clean Architecture**, especialmente separação de responsabilidades e direção adequada das dependências.

Não é exigida a reprodução rígida de uma arquitetura específica nem a criação artificial de camadas sem necessidade.

13.5. Processo de desenvolvimento e repositório

Serão considerados:

- histórico de desenvolvimento;
- existência de commits de todos os integrantes;
- distribuição coerente das contribuições;
- qualidade e clareza das mensagens de commit;
- consistência da evolução do projeto;
- qualidade e organização do repositório;
- uso adequado do `.gitignore`;
- ausência de artefatos gerados indevidamente versionados;
- qualidade da documentação;
- conformidade do `README.md` com o enunciado.

13.6. Etapa 2 — Sessão de apresentação, demonstração e arguição — até 5,0 pontos

A sessão de apresentação das equipes será realizada no dia **13/10/2026**.

Durante a sessão, **cada integrante receberá individualmente uma pontuação entre 0 e 5**, de acordo com o domínio demonstrado durante a apresentação e a arguição.

Poderão ser avaliados:

- compreensão das regras do jogo;
- tratamento de respostas constituídas por mais de uma palavra;
- tratamento e apresentação de espaços, hífen e outros caracteres não alfabéticos;
- restrição das tentativas às letras;
- compreensão da modelagem adotada;
- interpretação do código-fonte;
- justificativa das decisões técnicas;
- gerenciamento de estado;
- carregamento e desserialização do JSON;
- conversão de valores para tipos enumerados;
- normalização utilizada para ignorar acentuação;
- cálculo das estatísticas;
- organização arquitetural;
- capacidade de explicar ou modificar pequenos trechos da implementação.

Ausência de integrante

O integrante que **não comparecer à sessão de apresentação receberá pontuação 0** na sua participação individual.

Essa pontuação será utilizada normalmente no cálculo da média da equipe para a Etapa 2.

Cálculo da nota da apresentação

A **nota da apresentação da equipe** será calculada pela média aritmética das pontuações individuais de todos os seus integrantes, incluindo pontuação zero para integrantes ausentes.

O resultado da média será **arredondado para o número inteiro mais próximo**, produzindo uma nota entre **0 e 5**.

13.7. Composição da nota final

A nota final do trabalho será calculada pela soma das notas obtidas nas duas etapas:

Nota final = nota da entrega + nota da apresentação

em que:

- **nota da entrega:** valor entre 0 e 5;
- **nota da apresentação:** valor inteiro entre 0 e 5;
- **nota final:** valor entre 0 e 10.

14. Sessão de apresentação e verificação individual

Durante a sessão, qualquer integrante poderá ser solicitado a:

- demonstrar uma funcionalidade;
- explicar uma classe, método ou função;
- descrever uma regra do jogo;
- explicar como respostas compostas por várias palavras são representadas e processadas;
- explicar como espaços, hífen e outros caracteres são identificados e apresentados automaticamente;
- explicar por que esses caracteres não constituem tentativas do jogador;
- justificar uma decisão de implementação;
- identificar onde determinado requisito foi implementado;
- explicar um trecho desenvolvido originalmente por outro integrante;
- explicar o processo de carregamento e desserialização dos dados JSON;
- explicar a conversão de valores do JSON para tipos enumerados;
- explicar a normalização utilizada para ignorar acentuação;
- explicar o cálculo das estatísticas;
- justificar a organização arquitetural adotada;
- localizar responsabilidades específicas dentro do projeto;
- modificar ou complementar um pequeno trecho da solução.

Todos os integrantes deverão possuir conhecimento geral e técnico da solução entregue.

A sessão não se destina apenas à exposição ou demonstração visual do aplicativo. Seu objetivo central é verificar o **domínio individual sobre a solução implementada**, utilizando esse desempenho para compor a nota coletiva da apresentação.

15. Observações finais

O objetivo da atividade não é apenas produzir uma versão visualmente funcional do Jogo da Força.

A avaliação considerará também a capacidade da equipe de transformar regras conhecidas em uma solução de software:

- funcionalmente correta;
- capaz de tratar respostas formadas por uma ou mais palavras;
- capaz de apresentar corretamente espaços, hífen e demais caracteres estruturais sem exigir interação do jogador;
- adequadamente modelada;
- organizada;
- legível;
- responsiva;
- coerente com princípios de qualidade de software;
- tecnicamente justificável.

Decisões de interface e organização interna que não estejam explicitamente determinadas neste enunciado ficam sob responsabilidade da equipe, desde que não contrariem os requisitos apresentados.

O atendimento visual ou funcional aparente de um requisito não substitui a necessidade de uma implementação tecnicamente adequada e compatível com as regras estabelecidas neste enunciado.